

Week 2 “Supersonic” by Demi Kanon

Reference/Inspiration:

https://www.youtube.com/watch?v=SIThEI9IWXI&ab_channel=LoseControlMusic

- **Song:** “Supersonic” by Demi Kanon
 - The song has a vibrant, energetic feel that can inspire creative coding projects involving rhythm and motion. The dynamic beats and melodic shifts could serve as a reference for creating generative art based on music or designing projects that respond to audio input in real-time.
- **Inspiration:** Fusing electronic beats with an upbeat tempo is excellent for projects focused on sound-reactive visuals. This could guide work on generative visualizations where patterns or colours change in sync with the rhythm of the music.

Creative Coding Project or Plan:

- **Project Idea:** Create an interactive generative art piece in which visuals respond to the beats of “Supersonic.” This could involve using p5.js or Processing to create abstract visuals (like sound-reactive circles, waves, or particles) that pulse or morph based on the song’s rhythm and frequency analysis.
- **Challenges:** Synchronizing audio input with visual patterns might be challenging, especially for real-time responsiveness. It may require techniques like Fourier Transform for audio spectrum analysis and mapping the results to visual elements.

Tool, Technique, Library, or Tutorial to Explore:

- **Tool:** p5.js (or Processing)
 - It is an excellent library for creating generative art responding to music input. Specifically, p5.sound allows for audio analysis, which can be helpful in synchronizing visuals with sound. It will be essential for creating the generative, music-driven artwork you aim for.
- **Tutorial to Explore:** Look into “Audio Visualization with p5.js” tutorials, which show how to analyze audio files (like *Supersonic*) to generate real-time visuals. You can find relevant resources on the p5.js website or platforms like YouTube or GitHub.

Week 3 Three.js

Reference/Inspiration: <https://threejs-journey.com/>

Tool: Three.js

- Three.js is a powerful JavaScript library for creating 3D graphics in the browser. It simplifies working with WebGL, making it accessible for interactive, visually rich experiences. The inspiration comes from projects like the [Three.js journey](#), which showcases the library's capability for creating stunning generative art, immersive 3D environments, and interactive web-based visualizations. The ability to render 3D objects, incorporate real-time physics, and integrate dynamic lighting makes it an excellent tool for generative art and interactive applications.

Creative Coding Project or Plan:

- Build an interactive 3D web portfolio using Three.js.
- The portfolio will showcase my projects in a visually engaging 3D environment. Users can explore my work by navigating a dynamic scene.

Features to Include:

- A central 3D space (like a gallery or futuristic room) with clickable 3D objects representing different projects.
- Each object, when clicked, displays project details, links to GitHub, and visuals.
- Interactive camera movements and transitions to make navigation smooth and engaging.
- Use dynamic lighting and animations to bring the scene to life.

Tools & Techniques:

- Three.js for rendering the 3D environment.
- GSAP for smooth animations and transitions between scenes or project details.
- HTML/CSS to handle UI elements like menus or additional information.
- Textures and Models: Use tools like Blender or free 3D asset libraries for custom objects to represent projects.

Challenges:

- Designing a user-friendly navigation system for the 3D space.
- Optimizing performance for smooth interactions on various devices.
- Managing the balance between interactivity and usability—ensuring the portfolio remains functional and not just a visual experience.

Week 4 Art from Code

Reference/Inspiration: <https://art-from-code.netlify.app/day-1/session-1/>

Tool: Art from Code

- The tutorial introduces the fundamentals of generative art using code. It explores how simple loops, randomness, and basic shapes can create dynamic and visually engaging patterns. This session is a great starting point for understanding how to generate art programmatically.

Creative Coding Project or Plan:

- Plotting Data with Generative Art
- Use p5.js to create a visual representation of plotted data.
- Plot the data in a way that generates patterns and art.
- Experiment with different ways to map data points onto the canvas, using shapes, colours, and patterns inspired by generative art techniques.
- Implement randomness and interactivity to make the visualization more engaging.

Tools & Techniques:

- p5.js for drawing and rendering graphics.
- Loops and randomness to generate patterns dynamically.
- Data mapping to position elements based on numerical values.

Challenges:

- Finding a balance between randomness and structured data representation.
- Ensuring the visualization effectively communicates the data while still being aesthetically interesting.

Week 5 Boid Flocking

Reference/Inspiration: Concept: Boid Flocking Algorithm <https://eater.net/boids>

- Boid Flocking is a simple rule-based simulation of collective animal behaviour, such as bird flocks or fish schools. It is inspired by Craig Reynolds' 1986 "Boids" model, which demonstrates how local agent interactions can lead to complex, lifelike movement patterns. This concept is widely used in computer graphics, simulations, and generative art.

Creative Coding Project or Plan:

- Implement a boid simulation where particles (boids) move according to three primary rules:
 - Separation: Avoid crowding neighbours.
 - Alignment: Steer towards the average direction of nearby boids.
 - Cohesion: Move toward the center of the flock.
- Experiment with parameters like speed, field of view, and interaction strength.
- Add user interaction, allowing control over boid behaviour (e.g., clicking to attract or repel boids).

Tools & Techniques:

- p5.js for rendering and animation.
- Vector mathematics for steering behaviours.
- Loops and conditionals to handle interactions between multiple boids.
- Mouse or keyboard input to modify flocking behaviour dynamically.

Challenges:

- Balancing movement rules for realistic flocking behaviour.
- Optimizing performance when handling many boids at once.
- Fine-tuning parameters to create aesthetically pleasing motion.

Week 6 Laser Show Software

Reference/Inspiration: Laser Show Software

<https://lasershowprojector.com/blogs/education/laser-show-software?srsId=AfmBOooGE3XVENWW02TTNXF0gmuK5IXJGKnYlbMeRqXo98hPdkSvVZ3I>

- This week's inspiration comes from laser show software, which explores how lasers can be programmed to create dynamic light shows. These systems use specialized software to control beam movement, colour, and effects, enabling synchronized visual displays for music, performances, and interactive installations.

Creative Coding Project or Plan: Simulating Laser Patterns with Code

- Create a real-time laser effect simulation using p5.js or Three.js.
- Use mathematical functions (e.g., sine waves, Lissajous curves) to generate smooth, dynamic light patterns.
- Experiment with colour blending and motion trails to mimic laser effects.
- Add user controls to adjust speed, pattern complexity, or colour schemes.

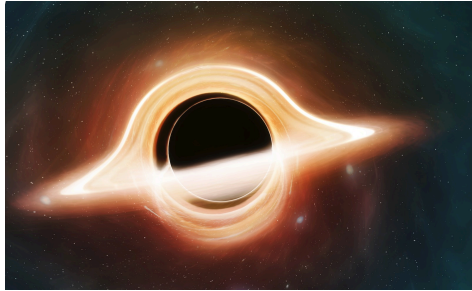
Tools & Techniques:

- p5.js or Three.js for rendering animated laser-like visuals.
- Sinusoidal equations to generate smooth wave-based motion.
- Colour transitions & gradients to simulate laser blending effects.
- User input (keyboard/mouse) to modify pattern settings dynamically.

Challenges:

- Achieving smooth and visually appealing laser motion.
- Simulating realistic light blending and glow effects.
- Optimizing performance to handle complex animations in real time.

Week 7 Black Hole Imagery



Reference/Inspiration: Black Hole Imagery

- This week's inspiration comes from an image of a black hole based on the Event Horizon Telescope. Black holes are fascinating cosmic objects that warp space and light, creating stunning gravitational lensing effects. The goal is to use code to simulate some of these visual distortions and motions inspired by real astrophysical phenomena.

Creative Coding Project or Plan: Black Hole Visualization

- Create a generative black hole simulation that mimics gravitational warping and light bending.
- Use particle motion to simulate matter swirling around an event horizon.
- Implement gravitational lensing effects to bend light around the black hole.
- Experiment with colour gradients to represent accretion disks and event horizon glow.

Tools & Techniques:

- p5.js or Three.js for real-time rendering.
- Perlin noise & distortion shaders to create a warped gravitational effect.
- Particle systems for a swirling accretion disk.
- Mathematical transformations to simulate curved spacetime.

Challenges:

- Creating a realistic distortion effect with smooth animations.
- Simulating light bending without heavy computations.
- Optimizing rendering for real-time performance.

Week 8 Colour Spectrum and City Visualization

Reference/Inspiration: The Colour Spectrum & Cityscapes

<https://easy-peasy.ai/cdn-cgi/image/quality=80,format=auto,width=700/https://media.easy-peasy.ai/a5d94604-0fb5-440d-b903-fec7a1fd2b22/781adeb1-004e-41b7-a0e1-45fff7fb2102.png>

- This project is inspired by the vibrant colour variations in urban environments, from neon-lit streets to natural daylight transitions. The idea is to blend the colour spectrum with a cityscape, creating a dynamic, abstract representation of a metropolis that shifts based on different lighting conditions.

Creative Coding Project or Plan: Dynamic Cityscape with Colour Spectrum

- Create a generative city skyline where buildings change colours based on a mapped spectrum.
- Implement time-based transitions, shifting colours from warm sunrise tones to cool night neon hues.
- Experiment with gradient blending and animated lights to simulate city vibrancy.
- Add interactive elements like mouse or keyboard input to control colour modes or building heights.

Tools & Techniques:

- p5.js or Three.js for rendering the cityscape.
- Colour interpolation to smoothly transition across the spectrum.
- Perlin noise or procedural generation for randomized but structured building layouts.
- Gradient shaders for smooth colour-blending effects.

Challenges:

- Balancing abstract colour effects with recognizable city forms.
- Creating smooth, visually appealing transitions between colour states.
- Optimizing performance for real-time colour shifts and animation.